Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Automatisation des traitements répétitifs et structures itératives en langage C

Synthèse exécutive : Ce cours pose les bases fondamentales des structures itératives (boucles) en algorithmique et programmation C. L'enjeu central est de comprendre l'automatisation des tâches répétitives sans duplication de code, en respectant le principe DRY (Don't Repeat Yourself). La séance détaille le fonctionnement, les critères d'évaluation des conditions (pré-condition vs post-condition) et la syntaxe C des trois types de boucles : 'while' (Tant Que), 'do-while' (Faire... Tant Que) et 'for' (Pour). Une attention particulière est portée sur la prévention des boucles infinies, la portée des variables d'itération et la rigueur syntaxique indispensable pour réussir les contrôles sur machine.

1. Utilité des structures itératives et principe DRY

En programmation, la répétition d'instructions ne doit jamais être gérée par copier-coller. Si un algorithme doit traiter 10 000 éléments d'un tableau ou redemander une saisie jusqu'à ce qu'elle soit correcte, dupliquer la ligne de code rend le programme lourd, illisible, impossible à maintenir et extrêmement vulnérable aux erreurs. Les structures itératives (ou boucles) permettent d'automatiser cette répétition en exécutant un bloc d'instructions tant qu'une condition logique (expression booléenne) reste vraie. Pour qu'une boucle fonctionne correctement et s'arrête, il est impératif que les instructions internes modifient l'état des variables intervenant dans la condition d'arrêt. Si la variable de contrôle n'est pas mise à jour, la condition reste indéfiniment Vraie, provoquant une 'boucle infinie' qui bloque l'exécution du programme et consomme 100% des ressources processeur.

- Principe DRY (Don't Repeat Yourself) : interdiction absolue de dupliquer du code pour exécuter plusieurs fois la même tâche.
- Condition d'arrêt : expression booléenne évaluée à chaque passage pour déterminer la poursuite ou l'interruption de la boucle.
- Variable de contrôle / Compteur : variable modifiée au sein du corps de la boucle pour faire évoluer la condition d'arrêt vers Faux.
- Boucle infinie : dysfonctionnement critique où la condition reste indéfiniment Vraie (souvent causée par l'oubli de la mise à jour du compteur).

Cas concret enseignant : *Le professeur a utilisé la métaphore du tapis roulant d'usine d'emballage : tant qu'un colis est détecté par le capteur (condition vraie), le bras robotisé l'attrape et l'emballe (instruction). Si le capteur ne détecte plus de colis ou si le quota fixé est atteint, le robot s'arrête. Sans condition de fin fonctionnelle, le robot continuerait à tourner à vide et finirait par surchauffer et détruire la machine.*

2. La boucle TANT QUE (while) : Structure à pré-condition

La structure 'while' est la boucle de référence lorsque le nombre exact d'itérations n'est pas connu à l'avance et dépend d'un événement dynamique externe (ex: une saisie utilisateur, la lecture d'un fichier jusqu'à la fin, ou la réception d'un signal réseau). En langage C, la syntaxe s'écrit : `while (condition) { /* instructions */ }` Puisque le test est réalisé avant toute exécution, si la condition s'avère fausse lors du premier contrôle, le corps de la boucle est totalement ignoré (0 itération). Il faut être extrêmement vigilant sur l'initialisation des variables de contrôle avant la boucle et leur modification explicite à l'intérieur du corps de la boucle pour éviter tout blocage indéfini.

- Pré-conditionnement : la condition booléenne est évaluée AVANT chaque exécution du bloc d'instructions.
- Plage d'itérations : de 0 à N passages. Si la condition est fausse dès l'entrée, le bloc d'instructions n'est jamais exécuté.
- Initialisation préalable : toutes les variables testées dans la condition du while doivent impérativement posséder une valeur définie avant d'entrer dans la boucle.

Cas concret enseignant : *Exemple du contrôle de code PIN au distributeur de billets : la machine demande le code TANT QUE le code saisi est incorrect ET QUE le nombre d'essais est inférieur à 3. Si l'usager tape le bon code dès le premier essai, la boucle s'arrête immédiatement sans répéter la demande.*

3. La boucle FAIRE... TANT QUE (do-while) : Structure à post-condition

À la différence de la boucle 'while', la structure 'do-while' exécute toujours les instructions de son corps une première fois avant de contrôler la condition logique de sortie. En langage C, la syntaxe s'écrit : `do { /* instructions */ } while (condition);` Elle est le choix algorithmique naturel pour les contrôles de saisie (validation de bornes numériques) et la gestion des menus interactifs, où une action doit être proposée au moins une fois avant de vérifier si l'utilisateur souhaite poursuivre. L'oubli du point-virgule final de la ligne `while (condition);` est l'une des fautes de syntaxe les plus fréquemment commises par les étudiants et bloque systématiquement la compilation.

- Post-conditionnement : le test logique est réalisé APRÈS l'exécution du bloc d'instructions.
- Garantie d'exécution : la boucle s'exécute obligatoirement au minimum 1 fois (intervalle de 1 à N itérations).
- Syntaxe C stricte : obligation impérative de placer un point-virgule ';' immédiatement après la parenthèse fermante de la condition while : `do { ... } while(condition);`

Cas concret enseignant : *Contrôle de saisie d'une note d'examen : 'Il est techniquement impossible de vérifier si la saisie faite par un étudiant est valide avant qu'il n'ait tapé au moins une valeur sur le clavier. On doit donc exécuter la saisie au moins une fois, ce qui fait du do-while la structure naturellement adaptée.'*

4. La boucle POUR (for) : Itération déterminée

La structure 'for' rationalise la mécanique itérative en réunissant les trois composantes fondamentales d'une boucle sur une seule ligne d'en-tête : 1. L'initialisation de la variable de contrôle (ex: `i = 0`). 2. La condition logique de maintien (ex: `i < n`). 3. La mise à jour/incrémentation (ex: `i++`). En langage C, la syntaxe s'écrit : `for (initialisation; condition; incrémentation) { /* instructions */ }` Bien que le langage C permette d'omettre certaines expressions dans l'en-tête (comme dans `for(;;) { ... }` qui crée une boucle infinie), les renseigner explicitement est une exigence absolue de lisibilité et de propreté algorithmique.

- En-tête condensé : centralise l'initialisation, la condition de maintien et la mise à jour sous la forme `for(init; cond; inc)`.
- Nombre d'itérations déterminé : réservée aux situations où le nombre exact de répétitions est connu avant d'entrer dans la boucle.
- Portée (scope) : possibilité de déclarer la variable de contrôle directement au sein de l'en-tête (ex: `for(int i = 0; ...)`).

Cas concret enseignant : *Métaphore des tours de piste d'athlétisme : 'Vous savez avant de vous lancer que vous devez effectuer exactement 5 tours. Vous initialisez votre compteur au tour 1 (init), vous vérifiez que vous n'avez pas dépassé 5 (cond), et à chaque franchissement de la ligne d'arrivée, vous ajoutez 1 à votre compteur (inc).'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
Réponse du professeur : Si vous n'incrémentez pas la variable 'i' dans le corps du while, la valeur de 'i' ne change jamais. La condition booléenne reste donc indéfiniment VRAIE. Le processeur réexécute en boucle le même bloc sans jamais atteindre la sortie, ce qui bloque totalement l'exécution du programme (boucle infinie).

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui, absolument. Sur le plan algorithmique, toute boucle 'for' peut être réécrite sous forme de boucle 'while' en plaçant l'initialisation avant la boucle et l'incrémentation à la toute fin du corps du 'while'. Cependant, on utilise 'for' par convention et pour la lisibilité lorsque le nombre d'itérations est connu dès le départ.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en langage C qui calcule la somme des N premiers entiers positifs (ex: si N = 5, calculer 1 + 2 + 3 + 4 + 5 = 15). Le programme doit valider que N est un entier strictement positif (> 0) au moyen d'une boucle 'do-while', puis calculer la somme cumulative à l'aide d'une boucle 'for'.

Résolution pas à pas :
Étape 1 : Déclarer les variables entières n, somme (initialisée à 0) et i. Étape 2 : Mettre en place la saisie sécurisée du nombre N avec la structure do-while : do { printf("Entrez N (>0) : "); scanf("%d", &n;); } while (n <= 0); Étape 3 : S'assurer que somme est initialisée à 0 avant le calcul (somme = 0). Étape 4 : Écrire la boucle for pour cumuler les valeurs de 1 jusqu'à N : for (i = 1; i <= n; i++) { somme += i; } Étape 5 : Afficher le résultat avec printf("La somme des %d premiers entiers est %d\n", n, somme);

Résultat : Pour un essai où l'utilisateur saisit d'abord -3 puis 5 : - La boucle do-while rejette -3 car (-3 <= 0) est VRAI, puis accepte 5 car (5 <= 0) est FAUX. - La boucle for réalise 5 itérations : * i=1 -> somme = 0 + 1 = 1 * i=2 -> somme = 1 + 2 = 3 * i=3 -> somme = 3 + 3 = 6 * i=4 -> somme = 6 + 4 = 10 * i=5 -> somme = 10 + 5 = 15 Le programme affiche correctement : 'La somme des 5 premiers entiers est 15'.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'évaluation entre une boucle 'while' et une boucle 'do-while' ?

R : La boucle 'while' évalue sa condition en pré-condition (avant l'exécution du bloc, soit de 0 à N itérations), alors que la boucle 'do-while' l'évalue en post-condition (après l'exécution du bloc, garantissant au moins 1 itération).

Q : Que se passe-t-il si la condition d'une boucle 'for' est fausse dès le premier test ?

R : Le corps de la boucle 'for' n'est jamais exécuté (0 itération), et le programme passe directement à l'instruction suivante.

Q : Quel oubli de syntaxe systématique provoque une erreur de compilation sur la boucle 'do-while' en langage C ?

R : L'oubli du point-virgule ';' obligatoire immédiatement après la parenthèse fermante de la condition du while : `do { ... } while(condition);`.